

Часть 2. Рабочий мини-инструмент и как он собирался

Что это и что оно делает. Я собрал доску заказов — один файл `index.html`, который открывается в браузере и работает офлайн, без сервера, установки и внешних библиотек. Заказ живёт в шести колонках: замер → проект → договор → производство → монтаж → сдача. Каждая карточка считает маржу автоматически (сумма минус себестоимость) и краснеет в двух случаях: если заказ висит на стадии дольше норматива или если прошла плановая дата сдачи, а акт ещё не подписан; жёлтая метка — рекламация, то есть заказ идёт, но убыток уже заложен. Сверху — срез по портфелю: сколько заказов просрочено и какая это доля, сколько стадий сверх норматива, сколько сорвано сроков, средняя маржа и число рекламаций. Данные лежат в браузере и в CSV — файл можно выгрузить, поправить в привычном Excel и загрузить обратно, разделители и форматы дат разбираются автоматически. Есть фильтры по дилеру, менеджеру, стадии и «только с проблемой», поиск и печать/PDF для планёрки. Ниже доски — блок аналитики, как в кейсах с дашбордами: структура портфеля кольцом, сумма и маржа по позициям, доля просрочки и рекламаций, динамика по месяцам и свод-таблица с подсветкой проблемных строк; вкладки переключают срез (дилеры, менеджеры, стадии), а фильтры сверху пересчитывают и графики, и таблицу. В демо-наборе 15 вымышленных заказов: 8 из 15 просрочены, средняя маржа 22,8 % — на таких цифрах уже можно разговаривать с производством и салонами, а не спорить о памяти.

Как я это собирал и где ломалось. Ставил задачу текстом и собирал с ИИ по шагам: сначала каркас доски и внешний вид, потом расчётная логика, потом загрузка и выгрузка. Главное правило — не верить цифрам на слово: я пересчитал вручную контрольные заказы, а потом написал автотест `selfcheck.js`, который вытаскивает расчётное ядро прямо из страницы и прогоняет 60 проверок (`node selfcheck.js`). Тест сразу поймал настоящую ошибку: в файле заказов колонка называется «Nº», нормализатор заголовков вырезал этот символ, столбец не находился — и все номера заказов подменялись на «Б/3», «Б/5». По скриншоту это было почти незаметно, по тесту — очевидно; починил, добавив распознавание «Nº» и синонимы заголовков. Дальше ломалось на живых файлах: Excel сохраняет CSV в cp1251, и русский текст превращался в кашу — добавил определение битой кодировки и повторное чтение в windows-1251; салоны пишут даты как 05.11.2026, а выгрузка идёт в ISO — парсер понимает оба формата, запятую как десятичный разделитель и точку с запятой как разделитель столбцов; незнакомая стадия не теряется, а добавляется отдельной колонкой с нормой по умолчанию. Часть правок была про аккуратность самого прототипа: сначала скрипт уехал внутрь блока стилей и страница осталась без оформления — переставил. Отфильтрованные срезы я сделал ссылками (например, `?hot=1` или `?dealer=Салон Казань`), чтобы руководителю можно было отправлять не скриншот, а живую ссылку на нужный вид. Аналитику собирал так же: графики нарисованы вручную в SVG, без внешних библиотек, — файл остаётся одним и открывается офлайн; вид подгонял под инфографику из кейса, а сами цифры на графиках проверяю тем же автотестом.

Прототип — живая доска заказов: https://petrobond.github.io/test-jobs/Systematica/case_operational_director/tool/ Исходники: `tool/index.html`, `tool/sample_orders.csv`, `tool/selfcheck.js`; скриншоты — в `screenshots/`.